

Serial link — communications guide

The drive speaks a small binary protocol over serial. `tools/hs300_protocol.py` is the **authority**: if this guide and that file disagree, the file is right.

Physical layer

baud	per drive - read BAUD_COM2. This plant's well drive is 9600
interface	RS485 (USB-RS485 adapter) or TTL (the ST-Link's USB serial)
framing	8N1
CRC	CRC16-CCITT, poly 0x1021, init 0xFFFF , over CMD..payload

`crc16("123456789")` must give `0x29B1` — use that to check any new implementation. **It is not the Modbus CRC**; the flow meter in `modbus_rtu.py` uses a different one. Mixing them up produces frames that are rejected without explanation.

The baud is a per-drive setting, not a constant. It lives in EEPROM as `BAUD_COM2` (and `BAUD_COM1`), an index into `{2400, 4800, 9600, 19200, 38400, 57600, 115200}`. The well drive here reads `BAUD_COM2 = 2`, which is **9600**.

This line used to say "9600 does not work on these builds, use 115200". That was wrong, and it cost an evening on 2026-10-03: every tool defaulted to 115200, every read came back as an ambiguous silence, and the blame went to the cable, the adapter, the bridge board and the drive's own power before anyone checked the baud.

You should not have to know. `open(verify=True)` pings for the firmware magic and, if nothing answers, scans the rest of the table and carries on at whatever replies, saying so. Every tool does this now. If a tool reports no drive, it has already tried every baud — believe it, and go and look at the drive.

In Python the port is opened as:

```
import hs300_protocol as P
L = P.Link('COM4', baud=115200, mode='rs485') # or mode='ttl'
L.open(verify=True) # pings, scans the bauds, raises if nothing is there

L = P.open_drive('COM4') # or just this: finds the baud by itself
```

Two register banks

bank	count	access
telemetry	167	read only — what the drive is doing
settings	75	read/write — how it is configured

Look settings up BY NAME, never by a remembered index:

```
L.read_telemetry() # the whole bank
L.read_settings()
L.write_setting(P.SET_BY_NAME['F09 FMAX'], 50.0)
L.command('STOP')
```

`P.SET_BY_NAME` and `P.TEL_BY_NAME` are the maps. Settings travel in **real units** over the wire — the panel's F-number and its $\times 10$ scaling are a separate thing, and confusing the two has caused real mistakes.

Telemetry is a snapshot

`comm_publish()` copies the live values into the register bank **once per 404 ms** UI slot. Polling faster returns the same numbers. 500 ms costs nothing and gains nothing over 400 ms.

So a value that looks frozen for a few hundred milliseconds is normal. A value frozen for **seconds** is not.

Writes do not persist by themselves

`write_setting()` changes the live value and the settings mirror. It does **not** touch the EEPROM. `CMD_SAVE_SETTINGS` is, in the firmware's own words, the only thing that writes it — 64 bytes, about 20 ms of blocked main loop, deliberately an explicit action.

To prove a value is really stored:

```
write_setting(...) -> CMD_SAVE_SETTINGS -> CMD_RELOAD_EEPROM -> read back
```

Reading straight after the write proves nothing.

The register map must match the firmware

If a telemetry register moves and the PC-side map does not, **the tools misread the drive silently** — no error, just wrong numbers. `COMM_FW_MAGIC` is bumped when the map changes so a stale tool is refused rather than believed.

There is no error for a *tool* that is behind. That is why `tools/` here is copied from the firmware tree on every publish instead of maintained separately: a second hand-kept copy once sat three map revisions behind and was blind to every register above 103.

`check_link_api.py` resolves every call every tool makes against the current map. `FAILURES: 0` means nothing is broken.

The capture recorder

The drive keeps a ring buffer — 3 channels × 256 samples — running from boot. Any protection trip freezes it, so the 51 ms leading *into* the trip is on the chip without anyone having armed anything.

```
python tools/hs300_capture_dump.py COM4 --no-arm --out trip.csv
```

Channels are selected by source id; `CAP_DIV` decimates, and the window is $256 \times \text{CAP_DIV} / 5000$ seconds.

Decimation is raw, not averaged, so anything above $2500/\text{CAP_DIV}$ Hz aliases into the record — fine for fast questions at `CAP_DIV = 1`, a trap for slow ones.

Two sources worth knowing, because the obvious ones mislead:

- 31 `vdc_inst_v` and 32 `idc_inst_a` are the **unaveraged** DC pair. VDC/IDC come from the RMS path and only update once per ~80 ms block, so a fast capture of those **stair-steps** — one trace showed 158 identical samples in a row — and any sample taken while the bus is moving is an average across a changing operating point.
- 12 `vDcInst` exists but is stored ×100 in a uint16, so it **clips at 655 V**. Useless on a bus that runs at 650–750 V. Use 31.

MQTT and the relay box

Separate device, separate repo. It uses the public broker `.hivemq.com` with the topic prefix `relay/<DEVICE_ID>/`.

There is no authentication on that path — the device id is the whole access control. Treat it as such.